TC631 OPERATING SYSTEM

Task 2: Thread Library in Linux and Windows

Converting a POSIX Threads Program to the Windows API and to C++ Standard Threads

Azriel Fajar Wicaksono

NIM 672025121

Instructor: Theophilus Wellem

Faculty of Information Technology

Satya Wacana Christian University

2026

1. Introduction

POSIX Threads (Pthreads) is the standard thread API on Linux and other Unix systems. A program uses it by including `<pthread.h>`. Windows does not use Pthreads. It has its own thread API, the **Win32 API** (`CreateThread`, `WaitForSingleObject`, and others). Because of this, the Pthreads example from the lecture cannot be compiled on Windows with Microsoft Visual C++.

The assignment is to convert the example program in two ways:

Program	Thread library	Language	Runs on
`0_pthread.c`	POSIX Threads (original lecture example)	C	Linux (Windows only through a translation layer)
`1_winapi_thread.c`	Windows API (Win32) threads	C	Windows only
`2_std_thread.cpp`	C++ standard thread library (`std::jthread`)	C++23	Windows and Linux, same code

All three programs do the same job in three steps: (1) the main thread creates one worker thread and passes it the number 1, (2) the main thread waits for the worker to finish, and (3) the main thread prints the value the worker returned ($1 \times 10 = 10$).

1.1 Environment

- Operating system: Windows 11
- Compiler: gcc (Rev13, Built by MSYS2 project) 15.2.0 (MinGW-w64, UCRT64 environment), used as `gcc` for C and `g++` for C++
- Language standard: C23 (`-std=c23`) for the C programs, C++23 (`-std=c++23`) for the C++ program

All outputs in this report are real outputs from running the programs. The script that builds this report compiles every program, runs each one five times, and checks that the worker always returns 10.

1.2 Why C++ and Not C for Program 2

The assignment allows either the C standard thread library (`<threads.h>`, C11) or the C++ standard thread library. I first tried C. On this compiler, `#include <threads.h>` fails with `fatal error: threads.h: No such file or directory`, because the MinGW-w64 C library does not include the C11 threads header. The C++ thread library is part of the compiler itself (libstdc++), so it works on Windows and Linux in the same way. The lecture slides also recommend `std::jthread` for C++20 and newer, so Program 2 uses C++23 with `std::jthread`.

2. The Original Program (POSIX Threads)

2.1 Source Code

This is the example from the lecture slides, unchanged. It is included so the two converted programs can be compared with it line by line.

0_pthread.c

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// Every thread function must return void* and accept void*
void* thread_runner(void* arg) {
    int* thread_num = (int*)arg; // Cast void pointer back to original type
    printf("Hello from Worker Thread #%d!\n", *thread_num);

    // Dynamically allocate a return value to safely pass back
    int* result = malloc(sizeof(int));
    *result = (*thread_num) * 10;

    pthread_exit((void*)result); // Equivalent to returning result;
}

int main() {
    pthread_t worker_thread;
    int thread_arg = 1;
    void* thread_return_val;

    // 1. Create the thread (ID, Attributes, Function, Argument)
    if (pthread_create(&worker_thread, NULL, thread_runner, &thread_arg) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the Main Thread!\n");

    // 2. Join the thread (Wait for it to finish and catch its return value)
    if (pthread_join(worker_thread, &thread_return_val) != 0) {
        perror("Failed to join thread");
        return 1;
    }

    // 3. Process the returned data
    int* output = (int*)thread_return_val;
    printf("Worker thread returned: %d\n", *output);

    free(output); // Clean up dynamically allocated memory
    return 0;
}
```

2.2 How the Program Works

1. **Lines 6–15,** `thread_runner`**:** a Pthreads thread function must take `void*` and return `void*`. The argument is cast back to `int*` (line 7). The return value must also be a pointer, so the result is stored in memory from `malloc` (lines 11–12) and its address is returned with `pthread_exit` (line 14).
2. **Line 23,** `pthread_create`**:** creates the thread and starts it. It returns 0 on success.
3. **Line 31,** `pthread_join`**:** waits for the thread to finish and receives the pointer it returned.
4. **Lines 37–40:** reads the result through the pointer and frees the memory the worker allocated.

2.3 Output

This program can still be run on Windows with MinGW, because MinGW includes **winpthreads**, a library that translates each Pthreads call into a Win32 call (option 2 on the "Options for Running Pthreads on Windows" slide). It is compiled with the `-pthread` flag:

```
Terminal (PowerShell)

> gcc -std=c23 0_pthread.c -o 0_pthread -pthread
> .\0_pthread.exe
Hello from the Main Thread!
Hello from Worker Thread #1!
Worker thread returned: 10
```

The output is the same as the output shown in the lecture slides. This is the result both converted programs must reproduce.

3. Program 1: Windows API Thread

3.1 Objective

Rewrite the program with the native Windows thread API, so it does not need `<pthread.h>` or any translation library. Every Pthreads call is replaced by its Win32 equivalent.

3.2 Source Code

1_winapi_thread.c

```c
// Program 1: Windows API thread
// The pthread example converted to native Win32 threads.
// Compile: gcc -std=c23 1_winapi_thread.c -o 1_winapi_thread

#include <windows.h>
#include <stdio.h>

// Every Win32 thread function must return DWORD and accept LPVOID (void*)
DWORD WINAPI thread_runner(LPVOID arg) {
    int* thread_num = (int*)arg; // Cast void pointer back to original type
    printf("Hello from Worker Thread #%d!\n", *thread_num);

    // The return value travels back as the thread's exit code (a DWORD),
    // so no malloc is needed
    DWORD result = (DWORD)(*thread_num) * 10;

    ExitThread(result); // Equivalent to return result;
}

int main(void) {
    HANDLE worker_thread;
    int thread_arg = 1;
    DWORD thread_return_val;

    // 1. Create the thread (Security, Stack size, Function, Argument, Flags, Thread ID)
    worker_thread = CreateThread(NULL, 0, thread_runner, &thread_arg, 0, NULL);
    if (worker_thread == NULL) {
        fprintf(stderr, "Failed to create thread (error %lu)\n", GetLastError());
        return 1;
    }

    printf("Hello from the Main Thread!\n");

    // 2. Join the thread (Wait until the thread handle is signaled = thread finished)
    if (WaitForSingleObject(worker_thread, INFINITE) != WAIT_OBJECT_0) {
        fprintf(stderr, "Failed to join thread (error %lu)\n", GetLastError());
        return 1;
    }

    // 3. Process the returned data (read the thread's exit code)
    if (!GetExitCodeThread(worker_thread, &thread_return_val)) {
        fprintf(stderr, "Failed to get exit code (error %lu)\n", GetLastError());
        return 1;
    }
    printf("Worker thread returned: %lu\n", thread_return_val);

    CloseHandle(worker_thread); // Clean up the thread handle
    return 0;
}
```

3.3 How the Program Works

1. **Line 5:** `<windows.h>` replaces `<pthread.h>`. The `-pthread` flag is no longer needed, because the thread functions live in `kernel32.dll`, which every Windows program already uses.

2. **Line 9, the thread function:** `CreateThread` requires a function of the form `DWORD WINAPI f(LPVOID)`. `LPVOID` is the Windows name for `void*`, `DWORD` is a 32-bit unsigned integer, and `WINAPI` sets the calling convention that Windows uses to call the function.
3. **Lines 10–11:** the same as the original. The argument is cast back to `int*` and the message is printed.
4. **Lines 15–17:** the result is calculated as a `DWORD` and the thread ends with `ExitThread(result)`, the Win32 version of `pthread_exit`. The value becomes the thread's **exit code**, which the operating system keeps inside the thread object.
5. **Line 21:** a `HANDLE` replaces `pthread_t`. A handle is a reference to a kernel object, in this case the thread object created by Windows.
6. **Line 26, `CreateThread`:** replaces `pthread_create`. It has six parameters:

Argument	Parameter	Meaning
`NULL`	Security attributes	Default security, and the handle cannot be inherited by child processes
`0`	Stack size	Use the default stack size set in the program file
`thread_runner`	Start address	The function the thread runs
`&thread_arg`	Parameter	The `void*` argument given to the function
`0`	Creation flags	Start running immediately (`CREATE_SUSPENDED` would create it paused)
`NULL`	Thread ID	The numeric thread ID is not needed here

7. **Lines 27–30:** `CreateThread` returns `NULL` on failure. Win32 functions report the reason through `GetLastError()`, not through `errno`, so `perror` from the original is replaced by `fprintf` with the error code.
8. **Line 35, `WaitForSingleObject`:** replaces the waiting part of `pthread_join`. A thread object becomes **signaled** when the thread ends. With the timeout `INFINITE`, the main thread sleeps until that happens, and the function returns `WAIT_OBJECT_0`.
9. **Line 41, `GetExitCodeThread`:** replaces the second part of `pthread_join`: it reads the exit code that the worker set on line 17.
10. **Line 47, `CloseHandle`:** releases the handle. Windows deletes the thread object only when the thread has ended **and** every handle to it is closed. Forgetting this call causes a handle leak. In Pthreads, `pthread_join` does this cleanup automatically, but in Win32 waiting and cleanup are two separate steps.

3.4 Output

```
Terminal (PowerShell)

> gcc -std=c23 1_winapi_thread.c -o 1_winapi_thread
> .\1_winapi_thread.exe
Hello from the Main Thread!
Hello from Worker Thread #1!
Worker thread returned: 10
```

3.5 Analysis

- The output is the same as the original Pthreads program, so the conversion is correct.
- **No `malloc` and `free`.** In Pthreads, a thread can only return a `void*`, so the original had to allocate memory, return its address, and free it in main. In Win32, the exit code is a number (`DWORD`), so a small integer like 10

can be returned directly. This removes the risk of a memory leak.

- **Limitation of the exit code:** it is only 32 bits. On 64-bit Windows a pointer is 64 bits, so a pointer cannot be returned this way. To return larger data, the thread should write it into a structure passed through the `LPVOID` argument. Also, the value 259 should not be used as a result, because `GetExitCodeThread` uses it as `STILL_ACTIVE` to mean "the thread is still running".
- "Hello from the Main Thread!" was printed first in every run. Creating a new thread takes some time, while main continues to its next line right away. This order is not guaranteed: the **operating system scheduler** decides which thread runs first, so on another machine the worker could print first. Only the last line is guaranteed to be last, because of `WaitForSingleObject`.

> Microsoft's documentation recommends `_beginthreadex` instead of `CreateThread` for threads that call the C runtime library (here, `printf`). The assignment asks for the Windows API thread, so `CreateThread` is used directly. With the modern Universal C Runtime used by this compiler, the program runs correctly, as the output shows.

3.6 Pthreads to Win32 Mapping

Step	Pthreads (original)	Windows API (Program 1)
Header	`<pthread.h>`	`<windows.h>`
Thread type	`pthread_t`	`HANDLE`
Function signature	`void* f(void*)`	`DWORD WINAPI f(LPVOID)`
Create	`pthread_create()`	`CreateThread()`
Terminate	`pthread_exit()`	`ExitThread()`
Wait for thread	`pthread_join()`	`WaitForSingleObject()`
Get return value	`pthread_join()` (second argument)	`GetExitCodeThread()`
Release resources	`pthread_join()` (automatic)	`CloseHandle()`
Error reporting	Return value is an error number	`NULL` / `FALSE`, then `GetLastError()`

4. Program 2: C++ Standard Thread Library

4.1 Objective

Rewrite the program with the C++ standard thread library, so the same source code compiles and runs on both Windows and Linux without any change.

4.2 Source Code

2_std_thread.cpp

```cpp
// Program 2: C++ standard thread library
// The pthread example converted to std::jthread (C++20/23).
// Compile: g++ -std=c++23 2_std_thread.cpp -o 2_std_thread

#include <thread>
#include <iostream>
#include <format>
#include <functional>

// A std::jthread function can have any parameters and types.
// The result is written into a reference, so no malloc and no void* casts.
void thread_runner(int thread_num, int& result) {
    std::cout << std::format("Hello from Worker Thread #{}!\n", thread_num);

    result = thread_num * 10; // Pass the return value back to main
}

int main() {
    int thread_arg = 1;
    int thread_return_val = 0;

    // 1. Create the thread (Function, Arguments). The thread starts immediately.
    //    std::ref passes thread_return_val by reference instead of copying it.
    //    If creation fails, std::system_error is thrown.
    std::jthread worker_thread(thread_runner, thread_arg, std::ref(thread_return_val));

    std::cout << "Hello from the Main Thread!\n";

    // 2. Join the thread (Wait for it to finish)
    worker_thread.join();

    // 3. Process the returned data
    std::cout << std::format("Worker thread returned: {}\n", thread_return_val);

    return 0; // Nothing to free: no heap memory, no handle to close
}
```

4.3 How the Program Works

1. **Lines 5–8:** `<thread>` provides `std::jthread`, `<format>` builds the output text, and `<functional>` provides `std::ref`.
2. **Line 12, the thread function:** unlike Pthreads and Win32, a C++ thread function can have any parameters with real types. Here it takes the thread number as an `int` and the place to store the result as an `int&` (a reference). No `void*` casting is needed, so the compiler can check the types.
3. **Line 13:** `std::format` builds the whole line first and prints it in one call. If the line were printed piece by piece with several `<<`, pieces from the two threads could get mixed in the same line.
4. **Line 15:** the result is written through the reference, directly into the `thread_return_val` variable of `main`.
5. **Line 25, creating the `std::jthread`:** replaces `pthread_create`. The thread starts immediately and runs `thread_runner(thread_arg, thread_return_val)`.

- By default, `std::jthread` **copies** every argument into the new thread. `thread_arg` is copied, which is safer than the original, where the worker read `main`'s variable through a pointer.
- `std::ref(thread_return_val)` wraps the variable so it is passed as a reference instead of a copy. Without `std::ref`, the worker would write into its own copy and `main` would never see the result.
- If the thread cannot be created, the constructor throws a `std::system_error` exception instead of returning an error code.

6. **Line 30, `join()`:** replaces `pthread_join`. Main waits until the worker is finished. `join()` also guarantees that everything the worker wrote is visible to main afterwards, so reading `thread_return_val` on line 33 is safe. Reading it before `join()` would be a data race.
7. **Line 35:** there is nothing to clean up. There is no heap memory to free and no handle to close. The `std::jthread` object releases the thread when it is destroyed at the end of `main` (RAII).

4.4 Output

```
Terminal (PowerShell)

> g++ -std=c++23 2_std_thread.cpp -o 2_std_thread
> .\2_std_thread.exe
Hello from the Main Thread!
Hello from Worker Thread #1!
Worker thread returned: 10
```

4.5 Analysis

- The output is the same as the original program and Program 1, so the conversion is correct.
- **Returning a value:** `join()` returns nothing, so a C++ thread cannot return a value the way `pthread_exit` does. This program writes the result into a reference instead. Two other standard ways are `std::promise` / `std::future` and `std::async`, which do return a value, but a reference is the simplest for one number.
- **`std::jthread` vs `std::thread`:** if a `std::thread` is destroyed without `join()`, the program crashes with `std::terminate`. A `std::jthread` ("joining thread", C++20) calls `join()` by itself in its destructor, so forgetting it cannot crash the program. `join()` is still written on line 30 so the waiting is clear and happens before the result is printed.
- **Portability:** this file uses only standard C++, so the same code compiles on Linux with `g++ -std=c++23` without changes. The compiler connects `std::jthread` to the operating system's own threads. On Linux it uses Pthreads. On this Windows compiler (thread model `posix`), it uses winpthreads, which uses Win32 threads underneath.
- As in Program 1, main printed first in every run, but the order of the first two lines is decided by the scheduler and is not guaranteed.

5. Comparison and Conclusion

	Pthreads (original)	Windows API (Program 1)	C++ `std::jthread` (Program 2)
Header	`<pthread.h>`	`<windows.h>`	`<thread>`
Thread type	`pthread_t`	`HANDLE`	`std::jthread`
Function signature	`void* f(void*)`	`DWORD WINAPI f(LPVOID)`	Any, e.g. `void f(int, int&)`
Create	`pthread_create()`	`CreateThread()`	`std::jthread` constructor
Wait	`pthread_join()`	`WaitForSingleObject()`	`join()`
Return value	Pointer to `malloc` memory	32-bit exit code	Written through a reference
Cleanup	`free()` the result	`CloseHandle()`	Automatic (destructor)
Errors	Return value, non-zero = error	`GetLastError()`	`std::system_error` exception
Runs on	Linux (Windows with winpthreads)	Windows only	Windows and Linux
Compile	`gcc -std=c23 0_pthread.c -o 0_pthread -pthread`	`gcc -std=c23 1_winapi_thread.c -o 1_winapi_thread`	`g++ -std=c++23 2_std_thread.cpp -o 2_std_thread`

The three programs give the same output, so both conversions are correct. They also show that the idea of a thread is the same on every system: **create** a thread, let it run at the same time as main, **wait** for it with join, and then read its **result**. Only the API is different. Pthreads and the Windows API are both low-level C APIs, tied to one operating system, that pass data as `void*` and need manual cleanup. The C++ standard thread library hides those details. It is portable, type-safe, and cleans up by itself, while underneath it still uses the operating system's own threads.

The part that surprised me most was how differently each API returns a value from a thread. Pthreads needs `malloc` and a pointer, Win32 uses the thread's exit code, and C++ does not return a value from `join()` at all, so the result has to go through a reference. I also learned that the Win32 version needs `CloseHandle`, because waiting for a thread and releasing it are two separate steps on Windows, while `pthread_join` does both.

References

1. T. Wellem, "TC631 Operating System, Assignment #2: Thread Library in Linux and Windows," lecture slides, Satya Wacana Christian University, 2026.
2. Microsoft Learn, "CreateThread function," "WaitForSingleObject function," "GetExitCodeThread function," and "CloseHandle function." Available: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/
3. cppreference.com, "std::jthread" and "std::ref." Available: https://en.cppreference.com/w/cpp/thread/jthread